

Esto es una prueba

Los asistentes

2026-01-30

Introducción

Los ficheros en quarto pueden tener varias extensiones, pero los específicos de quarto son ficheros `.qmd`.

Se puede escribir en markdown con algunas características especiales.

Para empezar a trabajar hay que instalar [quarto](#), y es recomendable instalar la correspondiente [extensión](#) en [vscode](#).

Una vez instalado quarto podemos crear proyectos o páginas a partir de proyectos desde la línea de comandos (con por ejemplo `quarto preview` o `quarto render`) o desde vscode.

En vscode pulsando `cmd+may+p` y tecleando `quarto` aparecerán muchas opciones relacionadas con quarto. Vamos a empezar por la más sencilla, la de crear un documento, que crea un documento con una cabecera como la de este fichero.

En esa cabecera podemos ir añadiendo campos, como autor, fecha, afiliación, si queremos tabla de contenidos, y muchas más opciones.

Algunas características propias de quarto

Bloques

En quarto se puede estructurar la información en bloques, que van delimitados por `:::` o más `:` si queremos anidar distintos bloques. Veremos ejemplos. a lo largo de este tutorial.

Centrando contenido

Podemos usar `<center>`

```
<center>
Esto está centrado
</center>
```

Esto está centrado

O bien un bloque

```
::: {style="text-align:center;"}
Centrado
:::
```

Centrado

Columnas

Empezamos un entorno con `{.columns}` y luego cada columna con un entorno `{.column}`.

Una columna

Otra columna

Podemos especificar el ancho de cada columna, por ejemplo con `{.column width="20%"}`.

Imágenes

Las imágenes se pueden insertar igual que en markdown, y al hacerlo, se crea una figura. El pie de foto, o leyenda, es el texto que viene entre corchetes. Las opciones van entre paréntesis.

Por ejemplo,

```
![Logo de la facultad](logo-ciencias.svg){width=20% fig-align="left" fig-alt="El logo de la
```

produce

O podemos combinar con centrado, produciendo



Figura 1: Logo de la facultad



Figura 2: Logo de la facultad

Bloques de llamada de atención

Por defecto hay cinco tipos de [llamadas](#)

- note
- warning
- important
- tip
- caution

Y se invocan en un bloque añadiendo `{.callout-tipo}`.

Prueba de `callout` a secas

Se pueden añadir distintas opciones.

Una nota

Esto es una nota “enrollable”.

Se pueden anidar

Tip

Prueba de `callout` a secas

Referencias cruzadas

Enlazo [aquí](#)

O bien usando la sintaxis @tipo-etiqueta: Figura 1 o bien imagen 1

Sección

Ocultando elementos dependiendo del tipo de documento

Podemos crear varios formatos del mismo documento precisándolo en la cabecera o bien usando

```
quarto render fichero.qmd --to pdf
```

Con el [contenido condicionado](#) podemos decirle a quarto qué contenido usar del documento dependiendo del tipo de documento que queramos.

La sintaxis para bloques enteros es

```
::: {.content-visible when-format="html"}
```

Sólo aparecerá si el documento es html.

```
:::
```

Para texto en línea usamos corchetes y luego entre llaves indicamos la condición.

```
Some text  
[en html.]{.content-visible when-format="html"}  
[en PDF.]{.content-visible when-format="pdf"}
```

Usando entornos virtuales

Hay varias formas de generar automáticamente figuras y salidas de código en python y R. Para ello tenemos que estar en un entorno del lenguaje de programación a usar. El lenguaje se especifica usando llaves.

```
```{python}  
print("Hola mundo")
```
```

También podemos utilizar [Observable JS](#). Por ejemplo,

```
1+1;
```